BI NEXUS

BUSINESS INTELLIGENCE PLATFORM

Feature Implementation Report

AI Report Generation & PDF Export

Document Type: Technical Feature Report

Date: April 15, 2026

Status: COMPLETED

Audience: Engineering Team / Stakeholders

Classification: Internal

1. Executive Summary

This report documents the implementation of the **AI Report Generation & PDF Export** feature for the BI Nexus platform. The feature extends the existing 7-step async data processing pipeline with an 8th step that uses Google Gemini to generate a department-specific narrative business report, and provides a PDF export endpoint so users can download a professionally formatted report.

8	7	2	8
NEW FILES CREATED	EXISTING FILES MODIFIED	NEW DEPENDENCIES	PIPELINE STEPS (WAS 7)

> **Key Outcome:** After uploading and processing a dataset, users can now view an AI-generated narrative business report directly in the browser and download it as a styled PDF document — all fully automated with zero manual effort.

2. Feature Overview

2.1 What Was Built

Two tightly integrated capabilities were added to BI Nexus:

1. **Step 8 — AI Report Generation:** A new pipeline step that runs after the Dashboard Blueprint (Step 7). It sends enriched metadata from all previous steps to Google Gemini, which generates a structured business report containing an Executive Summary, Key Insights, and Actionable Recommendations — tailored to the dataset's department (Sales, HR, Marketing, or Operations).
2. **PDF Export Endpoint:** A new REST API endpoint that renders the AI-generated report into a professionally styled PDF using WeasyPrint (with xhtml2pdf as a fallback), downloadable via a single button click from the frontend.

2.2 User Flow

```
Upload CSV/XLSX → Pipeline Steps 3-7 (existing) → Step 8: AI Report → View Report in Browser → Download as PDF
```

The entire flow is automatic. Once a file is uploaded and the pipeline completes, the report is available on the Report page (accessible from the sidebar and from a "View AI Report" button on the dashboard).

3. Technical Architecture

3.1 Backend Components

Component	File	Purpose
Step 8 Engine	`api/processing/step8_ai_report.py`	Constructs a detailed Gemini prompt from Steps 4-7 metadata, calls the API, parses the JSON response into structured sections, and generates raw HTML
Pipeline Integration	`api/tasks.py`	Added `_run_step8()` function, called after Step 7 and before row persistence. Stores result in `global_context.step8`
PDF Export View	`api/views.py`	New `export_pdf` endpoint: loads Step 8 data, populates the HTML template, renders to PDF via WeasyPrint/ xhtml2pdf
URL Route	`api/urls.py`	Added `datasets/<id>/export-pdf/` route
PDF Template	`api/templates/report_template.html`	Styled HTML template with header, department badge, sections, page numbers, and footer

3.2 Frontend Components

Component	File	Purpose
Report Page	`src/Components/Report/ReportPage.tsx`	Full-page report viewer: fetches Step 8 data via job status API, renders sections in styled cards, provides PDF download button
API Client	`src/api.js`	Added `exportPdf(datasetId)` function — POST request with `responseType: 'blob'`
Router	`src/App.tsx`	Added `/report` route
Sidebar	`src/Components/Sidebar/Sidebar.tsx`	Updated "Reports" link to point to `/report`

Component	File	Purpose
Dashboard	`src/Components/Hero/DashboardTemplate/DashboardTemplate.tsx`	Added "View AI Report" button (visible when Step 8 data exists)

3.3 Gemini AI Integration (Step 8)

The Step 8 prompt is constructed from rich metadata accumulated through the pipeline:

- **Dataset info:** file name, row count, column count, department category
- **Column profiles (Step 4-5):** data types, roles, semantic meanings, min/max/mean, null ratios
- **Data quality summary (Step 6):** rows before/after cleaning, transforms applied
- **Dashboard blueprint (Step 7):** suggested chart types with business justifications

Gemini returns a JSON object with exactly 3 sections: Executive Summary, Key Insights, and Recommendations. The system includes retry logic (strict JSON mode on failure) and a fallback report if the AI is unavailable.

Model Fallback Chain

```
gemini-2.5-flash  →  gemini-2.5-pro  →  gemini-3-flash-preview
```

If all models fail, a generic fallback report is generated programmatically.

3.4 PDF Generation Strategy

The system uses a two-tier PDF rendering strategy:

1. **Primary: WeasyPrint** — High-fidelity CSS rendering engine that produces professional-quality PDFs with proper page breaks, headers/footers, and font rendering. Requires GTK system libraries (installed via MSYS2 on Windows).
2. **Fallback: xhtml2pdf** — Pure Python PDF library with no system dependencies. Automatically used if WeasyPrint's GTK libraries are unavailable. Produces acceptable but lower-fidelity output.

> **Environment Note:** WeasyPrint requires MSYS2 and the `WEASYPRINT_DLL_DIRECTORIES` environment variable set to `C:\msys64\mingw64\bin` on Windows. This has been configured on the development machine. In production/CI, ensure GTK libraries are available or the xhtml2pdf fallback will be used.

4. Data Model

4.1 Step 8 Output Schema

Step 8 writes its output to `datasets.global_context.step8` with the following structure:

```
{
  "status": "completed",
  "generated_at": "2026-04-15T14:30:00+00:00",
  "department": "Sales",
  "report_html": "<h2>Executive Summary</h2>...",
  "sections": [
    { "title": "Executive Summary", "content": "..." },
    { "title": "Key Insights", "content": "..." },
    { "title": "Recommendations", "content": "..." }
  ]
}
```

4.2 API Endpoints

Method	Endpoint	Description	Auth
`POST`	`/api/datasets/{id}/export-pdf/`	Generates and returns the PDF report as a file download	Bearer token (Supabase JWT)
`GET`	`/api/check/{job_id}/`	Returns job status including `global_context.step8` when completed	Bearer token (Supabase JWT)

5. Files Changed

5.1 New Files

File	Lines	Description
`backend/api/processing/step8_ai_report.py`	~384	Step 8 engine: prompt builder, Gemini caller, JSON parser, HTML converter
`backend/api/templates/report_template.html`	~135	Styled HTML template for PDF rendering
`frontend/src/Components/Report/ReportPage.tsx`	~243	Full report viewer with PDF download

5.2 Modified Files

File	Change
`backend/api/tasks.py`	Added `_run_step8()`, updated pipeline to 8 steps
`backend/api/views.py`	Added `export_pdf` view, updated `TOTAL_PIPELINE_STEPS` to 8
`backend/api/urls.py`	Added export-pdf URL pattern
`backend/requirements.txt`	Added `weasyprint==63.1`, `xhtml2pdf==0.2.17`
`frontend/src/api.js`	Added `exportPdf()` function
`frontend/src/App.tsx`	Added `/report` route
`frontend/src/Components/Sidebar/Sidebar.tsx`	Updated Reports link
`frontend/src/Components/Hero/DashboardTemplate/DashboardTemplate.tsx`	Added "View AI Report" button
`frontend/src/Components/Hero/Hero.tsx`	Updated step count to 8, persist job ID to localStorage

6. Dependencies Added

Package	Version	Purpose	System Deps
`weasyprint`	63.1	Primary PDF renderer (CSS-compliant HTML-to-PDF)	GTK/Pango via MSYS2
`xhtml2pdf`	0.2.17	Fallback PDF renderer (pure Python)	None

System-Level Setup (Windows)

1. MSYS2 installed via `winget install MSYS2.MSYS2`
2. Pango + GTK libraries installed via `pacman -S mingw-w64-x86_64-pango` (38 packages)
3. Environment variable `WEASYPRINT_DLL_DIRECTORIES=C:\msys64\mingw64\bin` set at user level

7. Error Handling & Resilience

Scenario	Behavior
Gemini API unavailable	Tries 3 model variants in chain; if all fail, generates a generic fallback report
Gemini returns invalid JSON	Retries with strict JSON mode; applies JSON repair (smart quotes, trailing commas); falls back to generic report
WeasyPrint GTK libs missing	Automatically falls back to xhtml2pdf
xhtml2pdf also missing	Returns HTTP 500 with clear error message
No Step 8 data exists	Returns HTTP 400 instructing user to complete the pipeline first
Unauthorized request	Returns HTTP 401 — all endpoints require valid Supabase JWT

8. Known Limitations & Next Steps

8.1 Current Limitations

- The PDF contains text only — no charts or data visualizations are embedded
- The report is fixed at exactly 3 sections; users cannot customize the structure
- No caching: the PDF is regenerated on every download request
- xhtml2pdf fallback has lower CSS fidelity (some styling may differ)

8.2 Planned Enhancement PLANNED

A **PDF Report Overhaul** is planned as the next iteration, which will include:

- **Embedded chart images:** Server-side chart rendering via matplotlib, matching the dashboard's chart specs from Step 7
- **Cover page:** Branded first page with report title, department, dataset summary
- **KPI summary cards:** Visual metric cards showing key statistics
- **Improved layout:** Multi-page design with proper page breaks, two charts per page, and enhanced typography